

COMP101

Tutorial-07

Flowcharting

Flowcharting

- A flowchart is a graphical representations of steps
- Originated from computer science as a tool for representing algorithms and programming logic but had extended to use in all other kinds of processes
- Display information and assisting reasoning
- Help visualize complex processes, or make explicit the structure of problems and tasks
- Can be used to define a process or project to be implemented.

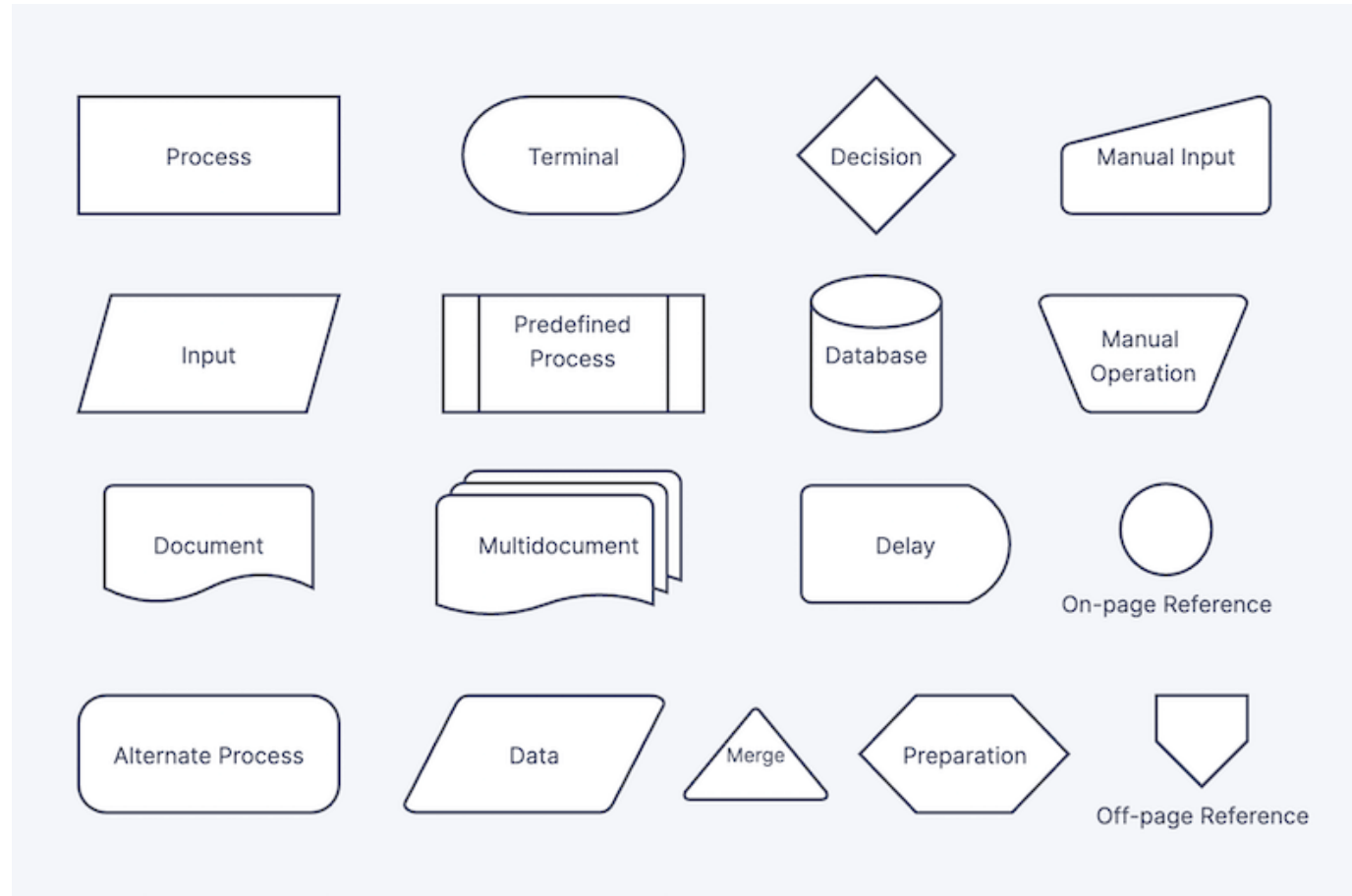
Flowcharting

- Flowcharts
 - 'flow diagram', 'work flow diagram'
- Represent workflow or process
- In programming represent an algorithm using standard symbols
- Established by John von Neumann (Princeton)
- Standards have been set for flowcharts by ANSI and ISO – although there is variation here

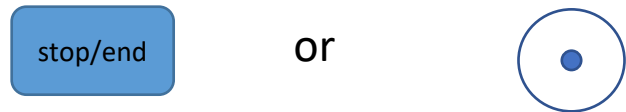
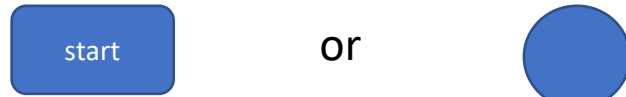
Flowcharting

- Use logical start and end points
- Use standard symbols
- Use simple decisions (based on T/F if possible)
- Flow is top to bottom, left to right
 - work left to right and top to bottom down the page
- Lines connect the symbols ('elements')
 - Avoid crossing lines
 - No arrows are needed unless this guidance is broken
- There must be only one start symbol and one stop symbol – 'terminal'

<https://www.zenflowchart.com/flowchart-symbols>



Terminals – only one of each in a chart



The four basic symbols most often used are terminals, rectangle, arrow and diamond
<https://www.gliffy.com/blog/guide-to-flowchart-symbols>

A process step is called an activity, e.g. calculations

A decision usually outputs as yes or no

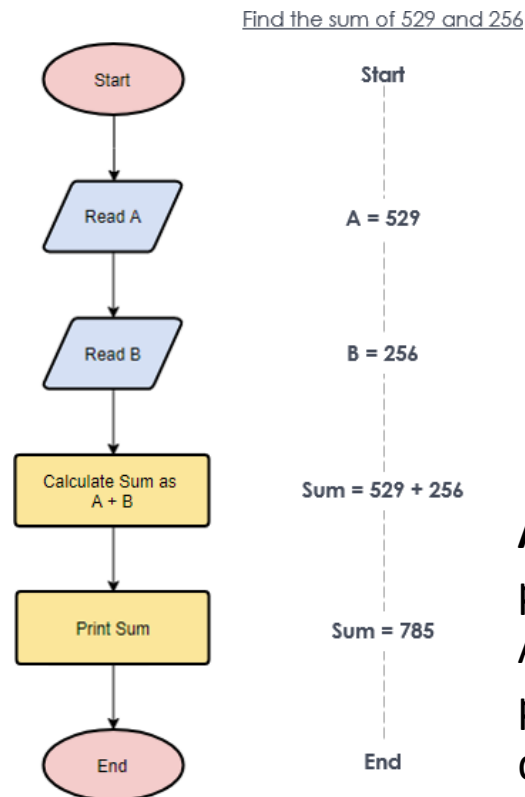
Arrows show relationships between the elements – use arrows only if needed

When to use a flowchart

- Helps understand processes
- Can improve processes
- Communication to peer group
- Flowchart procedure:
 - Define the boundaries (start and end)
 - Make the sequence logical
 - Keep it simple – usually shows you have the right logical approach to the problem
- Large programs can lend themselves to large diagrams – hard to follow
- Changing the design may have a knock-on effect for re-drawing the diagram

Visualise an algorithm - sequence

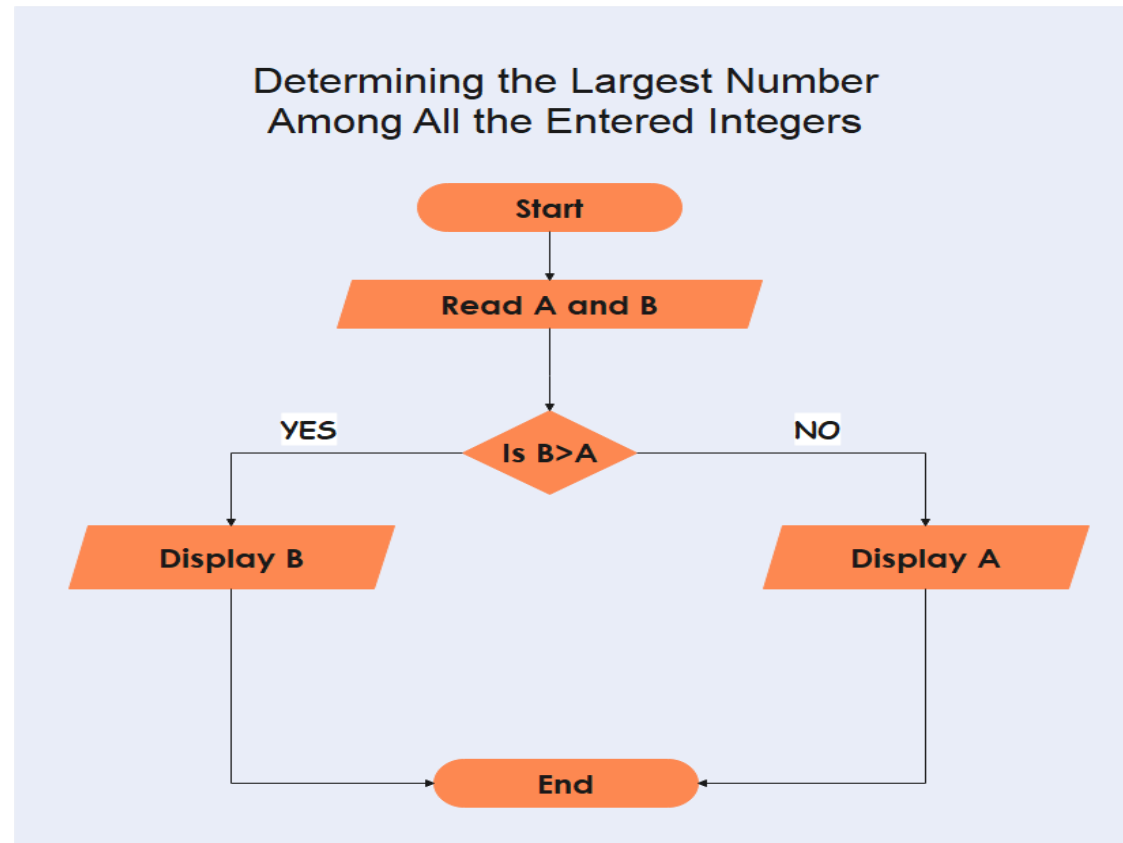
Sequential algorithm



Algorithms and **flowcharts** are two different ways of presenting the process of solving a problem. Algorithms consist of steps for solving a particular problem, while in flowcharts, those steps are usually displayed in shapes and process boxes with arrows. So flowcharts can be used for presenting algorithms

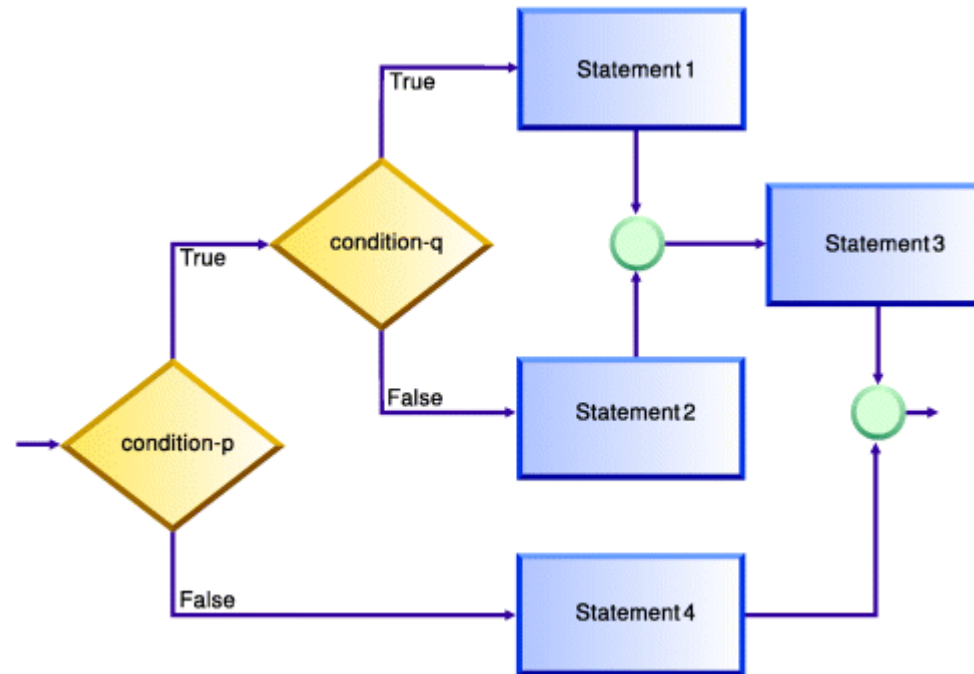
Visualise an algorithm - selection

Step 1: Read Integer A.
Step 2: Read Integer B.
Step 3: If B is greater than A,
then print B,
else A.



Nested if flowchart – check of logic

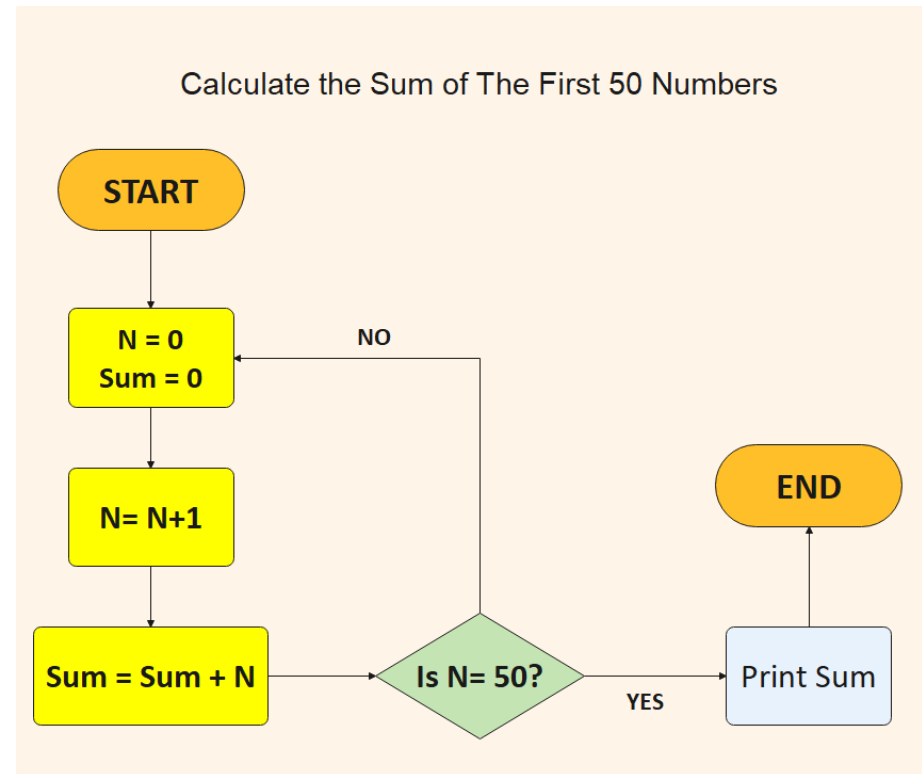
```
IF condition-p
  IF condition-q
    statement-1
  ELSE
    statement-2
END-IF
statement-3
ELSE
  statement-4
END-IF
```



<https://www.ibm.com/docs/en/cobol-zos/6.2?topic=actions-using-nested-if-statements>

Visualise an algorithm - iteration

Step 1: Declare number $N = 0$ and $\text{sum} = 0$
Step 2: Determine N by $N = N + 1$
Step 3: Calculate the sum by the formula:
 $\text{Sum} = N + \text{Sum}$.
Step 4: Add a loop between steps 2 and 3
until $N = 50$.
Step 5: Print Sum.



Visualise an algorithm

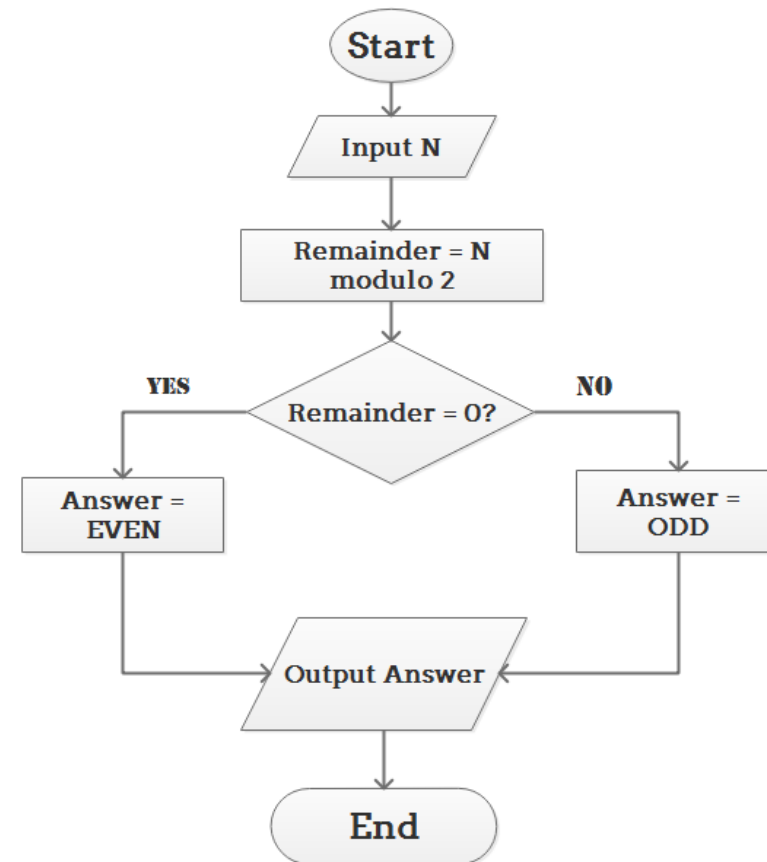
Determine and Output Whether Number N is Even or Odd

Step 1: Read number N.

Step 2: Set remainder as N modulo 2.

Step 3: If the remainder is equal to 0, then number N is even, else number N is odd.

Step 4: Print output.



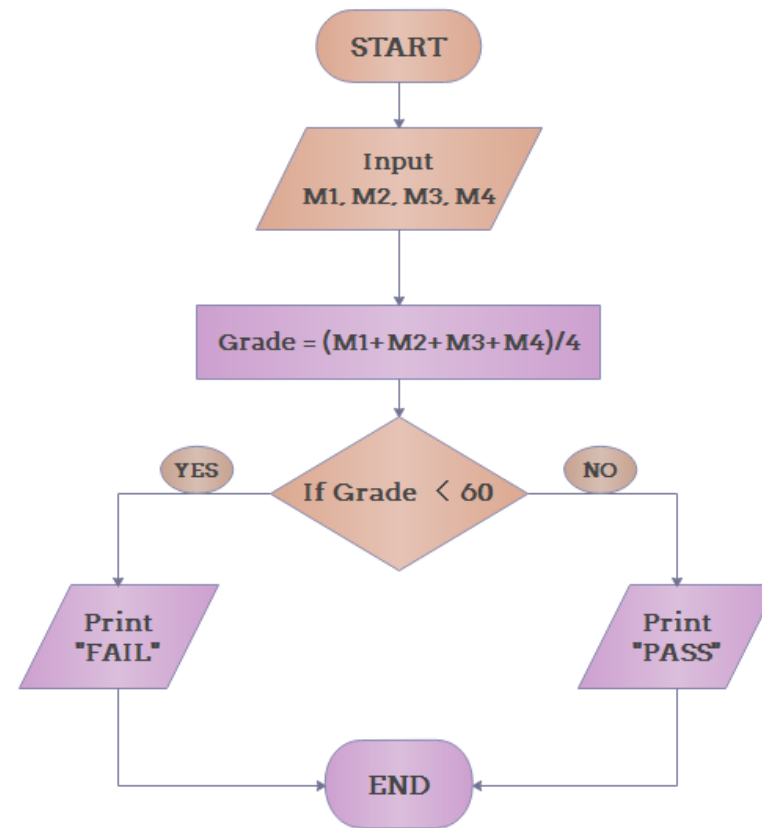
Visualise an algorithm

Determine Whether A Student Passed the Exam or Not

Step 1: Input grades of 4 courses M1, M2, M3 and M4.

Step 2: Calculate the average grade with the formula " $\text{Grade} = (\text{M1} + \text{M2} + \text{M3} + \text{M4}) / 4$ ".

Step 3: If the average grade is less than 60, print "FAIL", else print "PASS".



Compare and contrast flowcharts with pseudocode and code

- Flowcharts are good for visualizing and communicating algorithms
 - can be cumbersome and time-consuming to create and maintain
- Pseudocode is good for planning and testing algorithm
 - can be ambiguous and inconsistent
 - ... even though it improves on natural language
- Code is good for implementing and optimizing algorithms
 - can be hard to read and debug.

Compare and contrast flowcharts with pseudocode and code

- Look at their level of abstraction
 - Abstraction - process of simplifying and generalizing complex details and concepts
- Flowcharts are the most abstract
 - they use symbols and text to represent the main steps and logic of an algorithm
- Pseudocode is less abstract
 - uses a subset of natural language and some programming concepts to describe the details and logic of an algorithm
- Code is the least abstract
 - it uses specific rules and syntax to implement the exact instructions and logic of an algorithm.

<https://www.linkedin.com/advice/0/how-do-you-compare-contrast-flowcharts-pseudocode>

Compare and contrast flowcharts with pseudocode and code

- Depending on goals and preferences, use flowcharts, pseudocode, and code in different stages and ways in your algorithm development process
- Use flowcharts to brainstorm and sketch your initial ideas
- Use pseudocode to refine and test your logic, and code to implement and optimize your solution
- Or you can use pseudocode as a bridge between flowcharts and code, or code as a way to verify and improve your flowcharts or pseudocode
- Use the method that suits your needs and style
 - Be consistent and clear in representation and communication
- Tools and resources that can help you create and convert flowcharts, pseudocode, and code easily and efficiently: Use online editors, generators, converters, or libraries to create flowcharts, write pseudocode, or generate code from flowcharts or pseudocode

<https://www.linkedin.com/advice/0/how-do-you-compare-contrast-flowcharts-pseudocode>

If A:

B

D

if D:

E

if E:

G

else

F

if F:

G

else

C

else

F

if F:

G

else

C

else

C

F

if F:

G

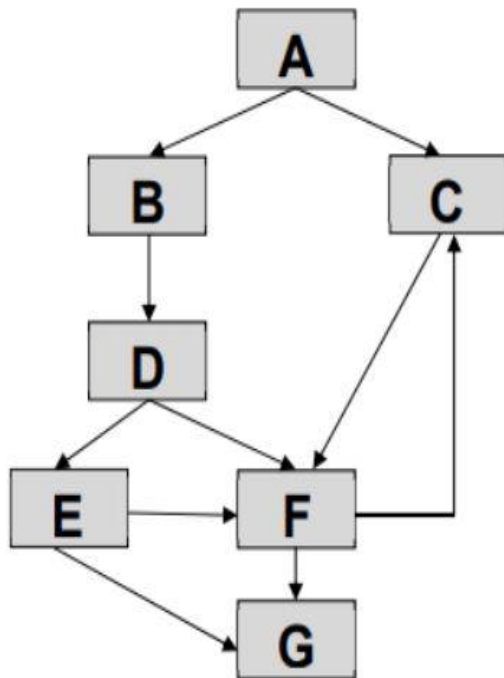
else

C

Flowchart Testing

Draw the flowchart

Flowchart testing



In the diagram, there are four conditions/decisions: A, D, E, F

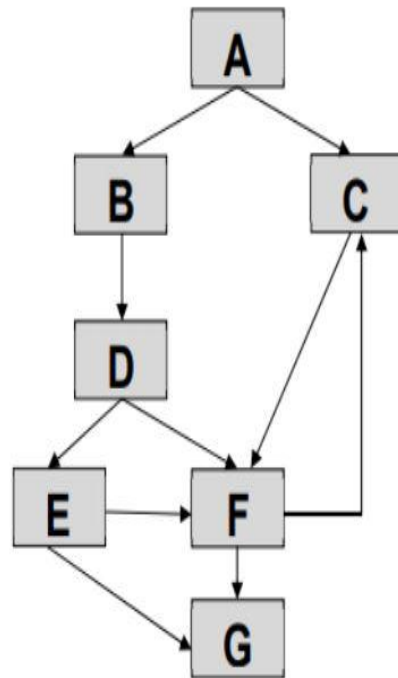
One of the most important test goals is 100% decision coverage.
i.e., we employ tests for all the paths through the chart.

Test01 covers path: A, B, D, E, G

Test02 covers path: A, B, D, E, F, G

Test03 covers path: A, C, F, C, F, C, F, G

Flowchart testing



In the diagram, there are four conditions/decisions: A, D, E, F

Test_01 covers A->B, D->E and E->G;

path: A, B, D, E, G

Test_02 covers A->B, D->E, E->F and F->G;

path: A, B, D, E, F, G

Test_03 covers A->C, F->C and F->G.

path: A, C, F, C, F, C, F, G

Condition A covered:

A->B by Test_01 and A->C by Test_03;

Condition E covered:

E->G by Test_01 and E->F by Test_02;

Condition F covered:

F->C by Test_03 and F->G by Test_02 and Test_03;

Condition D is not covered:

only D->E by Test_01 and Test_02, but D->F is not covered.

?flowchart testing

- In the diagram, there are four conditions/decisions: A, D, E, F
- The Test_01 covers A->B, D->E and E->G
- Test_02 covers A->B, D->E, E->F and F->G
- The Test_03 covers A->C, F->C and F->G.
- Hence coverage in case of condition A is complete (A->B by Test_01 and A->C by Test_03),
- condition E is covered (E->G by Test_01 and E->F by Test_02),
- Covered condition-F (F->C by Test_03 and F->G by Test_02 and Test_03).
- The no coverage of Condition D, there is only D->E by Test_01 and Test_02, but D->F is not covered.